

Dataset Annotation

Dataset Annotation

- 1. Dataset Collection
 - 1.1 Extracting Images
 - 1.2 Dataset Path
- 2. Using Label Studio
 - 2.1 Installing Label Studio
 - 2.2 Starting Label Studio
 - 2.2.3 Accessing Label Studio
 - Host Machine
 - Same Local Area Network
 - First-Time Use
- 3. Dataset Annotation
 - 3.1 Create Project
 - 3.2 Project Name
 - 3.3 Import Training Set
 - 3.4 Label Settings
 - 3.5 Dataset Annotation
 - 3.6 Export Dataset
 - 3.7 Dataset Directory Structure
 - 3.8 Dataset Configuration File
- References

For recognizing specific objects or improving object recognition accuracy, users need to train the model themselves. Dataset collection and annotation are indispensable parts of this process.

1. Dataset Collection

Description: For recognition in specific scenarios, users can use a camera to record videos of objects from various angles, then extract frames from the video as the training set.

1.1 Extracting Images

A simple example: Extract one frame from the video every 15 frames as the dataset.

```
import cv2
import os

# Path to the input video file
video_file = '/home/sunrise/ultralytics26/data/yahboom_data/orange_video/orange.mkv'

# Path to the output folder to save the frames
output_folder = '/home/sunrise/ultralytics26/data/yahboom_data/orange_pic'

# Check if the output folder exists, if not, create it
if not os.path.exists(output_folder):
    os.makedirs(output_folder)

# Open the video file using OpenCV
```

```

cap = cv2.VideoCapture(video_file)

# Get the total number of frames in the video
total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

# Get the frames per second (fps) of the video
fps = cap.get(cv2.CAP_PROP_FPS)

# Frame counter to keep track of the current frame number
frame_number = 0

# Define the interval for extracting frames (every 15th frame)
frame_interval = 15

# Initialize a new counter for saved frames (starting from 1)
saved_frame_number = 1

while True:
    ret, frame = cap.read()

    # If the frame is not successfully read, exit the loop
    if not ret:
        break

    # Save every 15th frame
    if frame_number % frame_interval == 0:
        # Format the frame number starting from 1 (e.g., 1.png, 2.png)
        image_name = f'{saved_frame_number}.png'
        image_path = os.path.join(output_folder, image_name)

        # Save the current frame as a PNG image
        cv2.imwrite(image_path, frame)
        print(f'Saving frame {frame_number}/{total_frames} as {image_name}')

        # Increment the saved frame counter
        saved_frame_number += 1

    # Increment the frame counter for the video
    frame_number += 1

# Release the video capture object
cap.release()

# Print message when processing is complete
print("Video processing complete! Every 15th frame saved as an image!")

```

1.2 Dataset Path

Example video: /home/sunsie/ultralytics26/data/yahboom_data/orange_video

Example images: /home/sunrise/ultralytics26/data/yahboom_data/orange_pic

2. Using Label Studio

Label Studio is an open-source data annotation platform used for data labeling and annotation task management, supporting various types of data input and annotation formats.

2.1 Installing Label Studio

```
pip install label-studio -i https://mirrors.aliyun.com/pypi/simple/ --trusted-host mirrors.aliyun.com --break-system-packages
```

2.2 Starting Label Studio

Enter in terminal:

```
label-studio start
```

```
sunrise@ubuntu:~$ label-studio start
=> Database and media directory: /home/sunrise/.local/share/label-studio
=> Static URL is set to: /static/
=> Database and media directory: /home/sunrise/.local/share/label-studio
=> Static URL is set to: /static/
Read environment variables from: /home/sunrise/.local/share/label-studio/.env
get 'SECRET KEY' casted as '<class 'str'>' with default ''
Starting new HTTPS connection (1): pypi.org:443
https://pypi.org:443 "GET /pypi/label-studio/json HTTP/1.1" 200 36245
/home/sunrise/.local/lib/python3.10/site-packages/google/api_core/python_version_support.py:275: FutureWarning: You are using a Python version (3.10.12) which Google will stop supporting in
new releases of google.api_core once it reaches its end of life (2026-10-04). Please upgrade to the latest Python version, or at least Python 3.11, to continue receiving updates for google
api_core past that date.
  warnings.warn(message, FutureWarning)
/home/sunrise/.local/lib/python3.10/site-packages/django/db/backends/utils.py:98: RuntimeWarning: Accessing the database during app initialization is discouraged. To fix this warning, avoid
executing queries in AppConfig.ready() or when your app modules are imported.
  warnings.warn(self.APPS_NOT_READY_WARNING_MSG, category=RuntimeWarning)
March 26, 2026 - 08:02:11
Django version 5.1.15, using settings 'label_studio.core.settings.label_studio'
Starting development server at http://0.0.0.0:8080/
Quit the server with CONTROL-C.

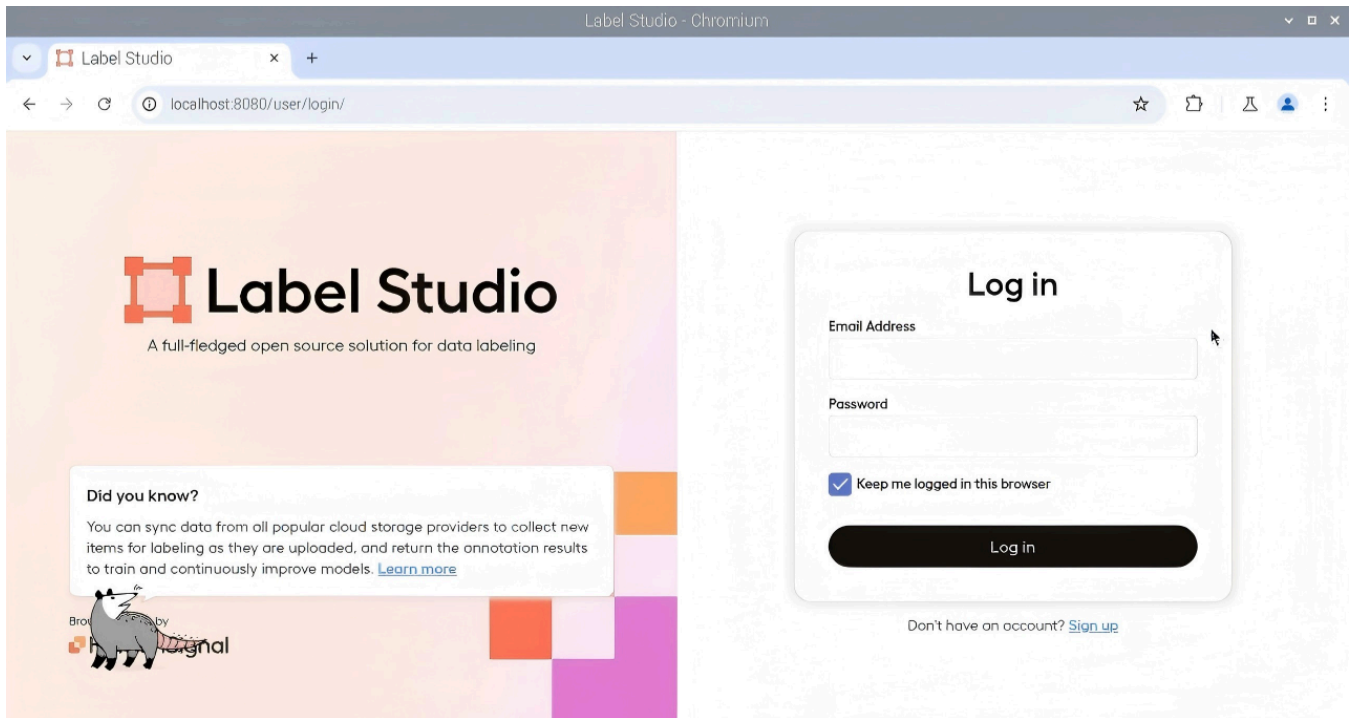
(exo-open:25004): dbind-WARNING **: 08:02:14.471: AT-SPI: Error retrieving accessibility bus address: org.freedesktop.DBus.Error.ServiceUnknown: The name org.ally.Bus was not provided by an
.service files
(xfce4-mime-helper:25006): dbind-WARNING **: 08:02:14.593: AT-SPI: Error retrieving accessibility bus address: org.freedesktop.DBus.Error.ServiceUnknown: The name org.ally.Bus was not provi
ed by any .service files
[2026-03-26 08:02:16.942] [django.server::log_message:213] [INFO] "GET /sw.js HTTP/1.1" 200 6221
[2026-03-26 08:02:16.942] [django.server::log_message:213] [INFO] "GET /sw.js HTTP/1.1" 200 6221
[2026-03-26 08:02:17.125] [django.server::log_message:213] [INFO] "GET / HTTP/1.1" 200 38973
[2026-03-26 08:02:17.125] [django.server::log_message:213] [INFO] "GET / HTTP/1.1" 200 38973
[2026-03-26 08:02:17.743] [django.server::log_message:213] [INFO] "GET /static/fonts/Figtree-Regular.ttf HTTP/1.1" 200 40584
[2026-03-26 08:02:17.743] [django.server::log_message:213] [INFO] "GET /static/fonts/Figtree-Regular.ttf HTTP/1.1" 200 40584
[2026-03-26 08:02:17.757] [django.server::log_message:213] [INFO] "GET /react-app/runtime.js?v=2a9bfb HTTP/1.1" 200 5034
[2026-03-26 08:02:17.757] [django.server::log_message:213] [INFO] "GET /react-app/runtime.js?v=2a9bfb HTTP/1.1" 200 5034
[2026-03-26 08:02:17.760] [django.server::log_message:213] [INFO] "GET /static/fonts/Figtree-SemiBold.ttf HTTP/1.1" 200 40852
[2026-03-26 08:02:17.760] [django.server::log_message:213] [INFO] "GET /static/fonts/Figtree-SemiBold.ttf HTTP/1.1" 200 40852
[2026-03-26 08:02:17.825] [django.server::log_message:213] [INFO] "GET /react-app/main.css?v=2a9bfb HTTP/1.1" 200 232558
[2026-03-26 08:02:17.825] [django.server::log_message:213] [INFO] "GET /react-app/main.css?v=2a9bfb HTTP/1.1" 200 232558
```

2.2.3 Accessing Label Studio

Host Machine

For RDX S600 series development board access method, first-time users need to enter their own username and password.

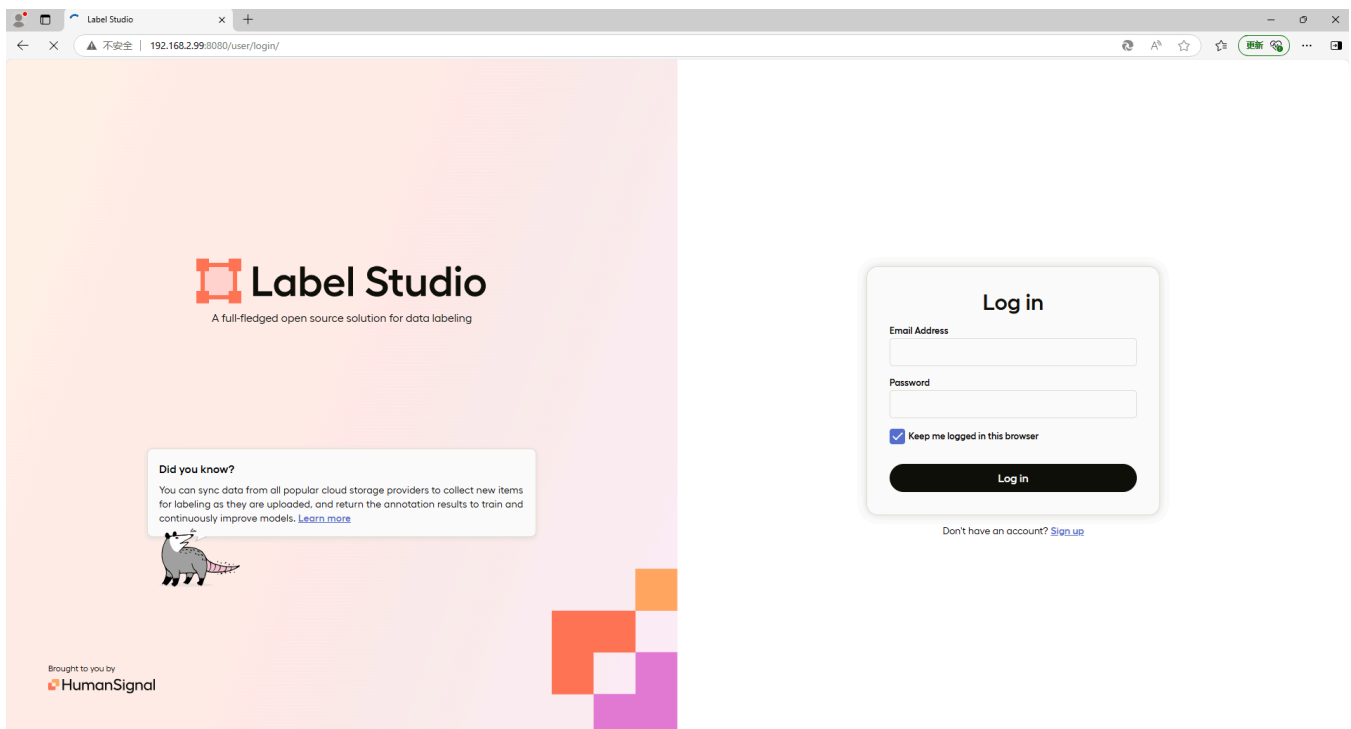
```
http://localhost:8080/
```



Same Local Area Network

Devices on the same local area network can access via the board IP:8080.

`http://Board IP:8080/ # For example: http://192.168.2.99:8080/`



First-Time Use

First-time users need to register and login. Factory default account information (account information is fictional):

Account: yahboom@163.com
Password: yahboom@163.com

Label Studio - Chromium

Label Studio

localhost:8080/user/login/

Label Studio

A full-fledged open source solution for data labeling

Did you know?

You can sync data from all popular cloud storage providers to collect new items for labeling as they are uploaded, and return the annotation results to train and continuously improve models. [Learn more](#)

Brooklyn by Signal

Log in

Email Address

Password

☒ Keep me logged in this browser

Log in

Don't have an account? [Sign up](#)

Label Studio - Chromium

Label Studio

localhost:8080/user/signup/

Label Studio

A full-fledged open source solution for data labeling

Did you know?

Label Studio now has a Starter Cloud offering optimized for small teams and projects. [Learn more](#)

Brooklyn by Signal

Sign Up

Email Address

yahboom@163.com

Password

How did you hear about Label Studio?

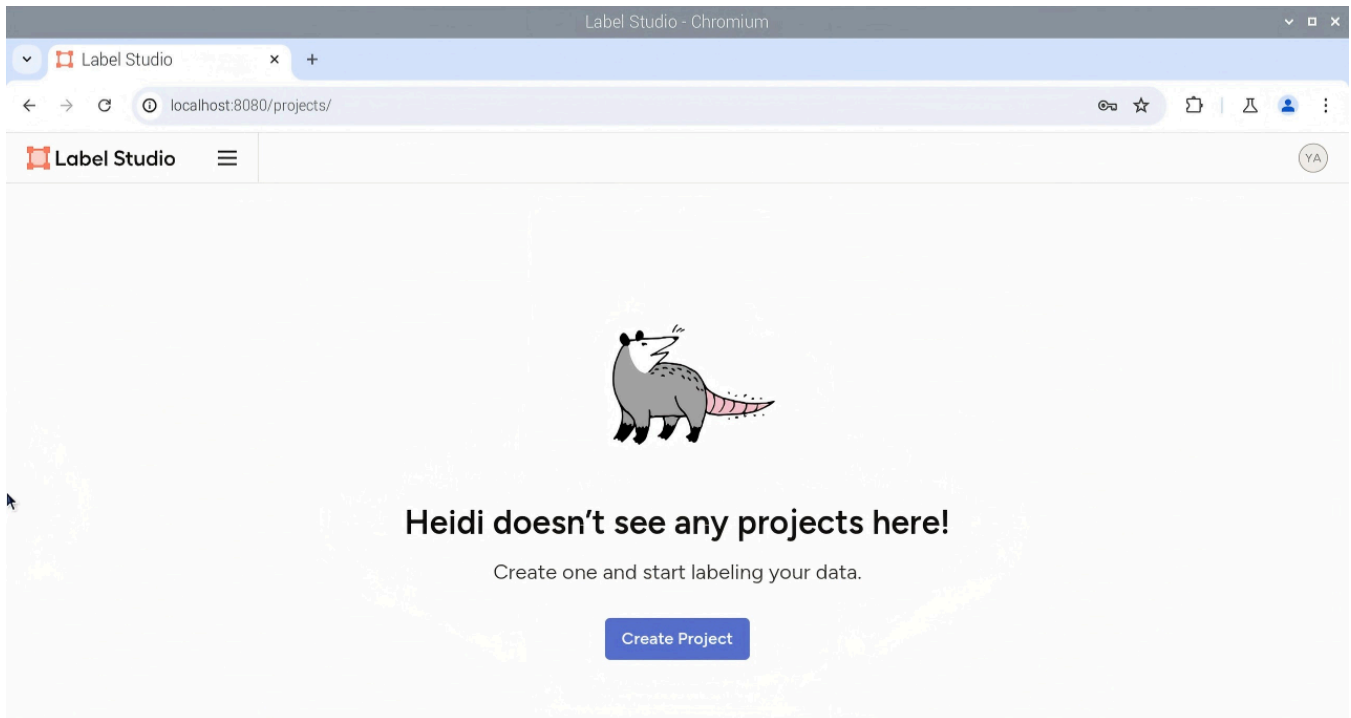
Github

☐ Get the latest news from Heidi

Create Account

Already have an account? [Log in](#)

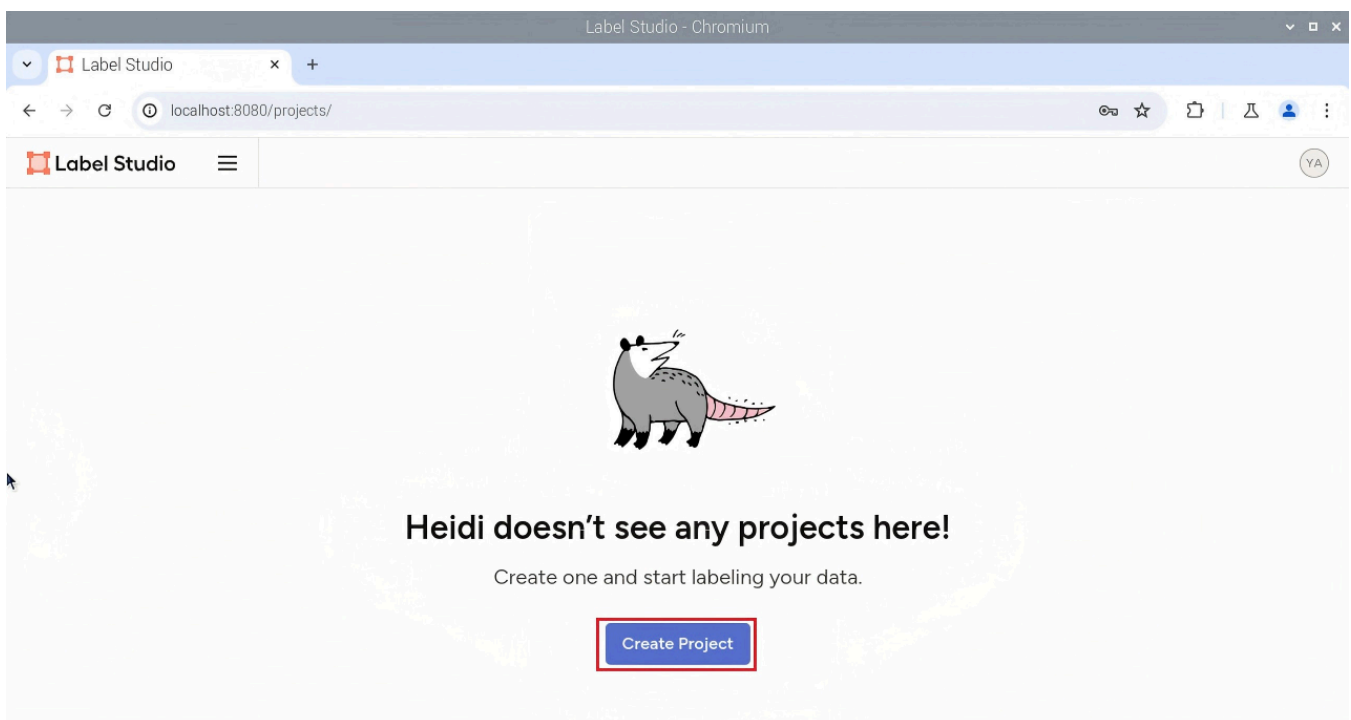
After registration, the website will automatically log in:



3. Dataset Annotation

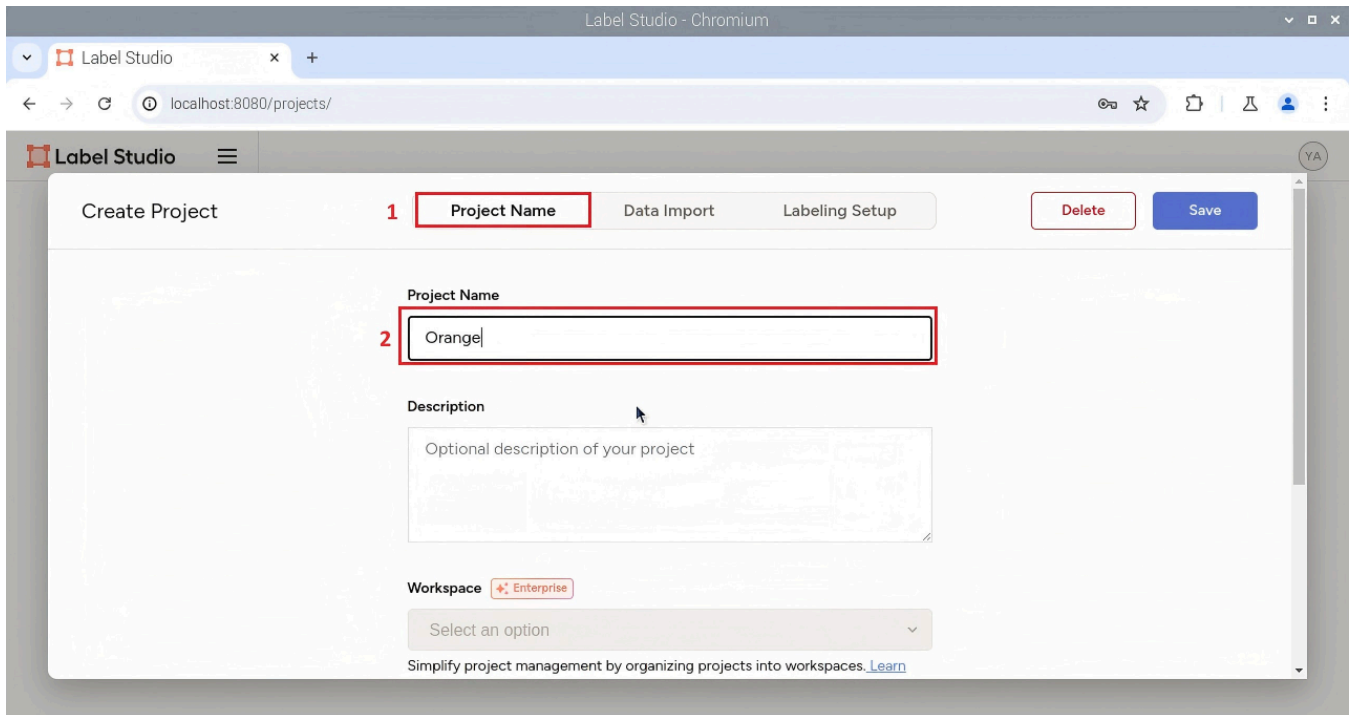
The demonstration dataset is on the development board, so access Label Studio from the board.

3.1 Create Project



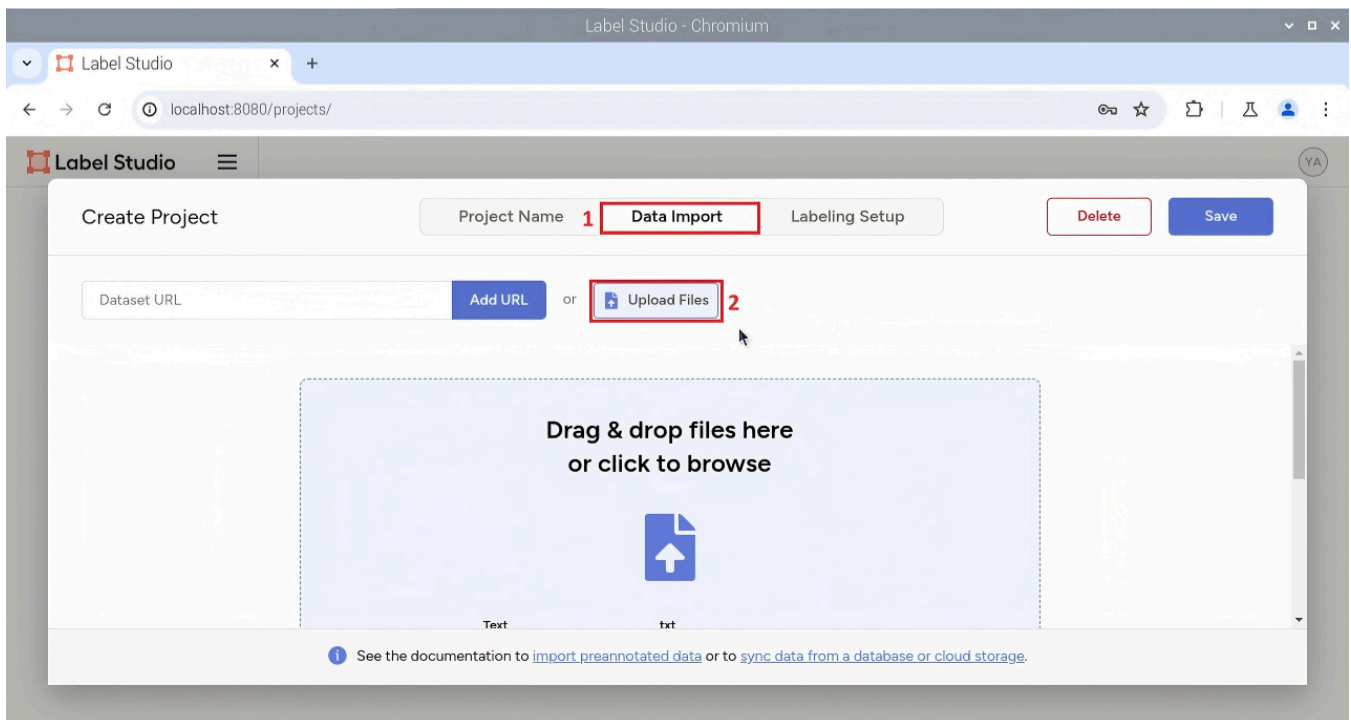
3.2 Project Name

The project name can be freely chosen based on your training set:



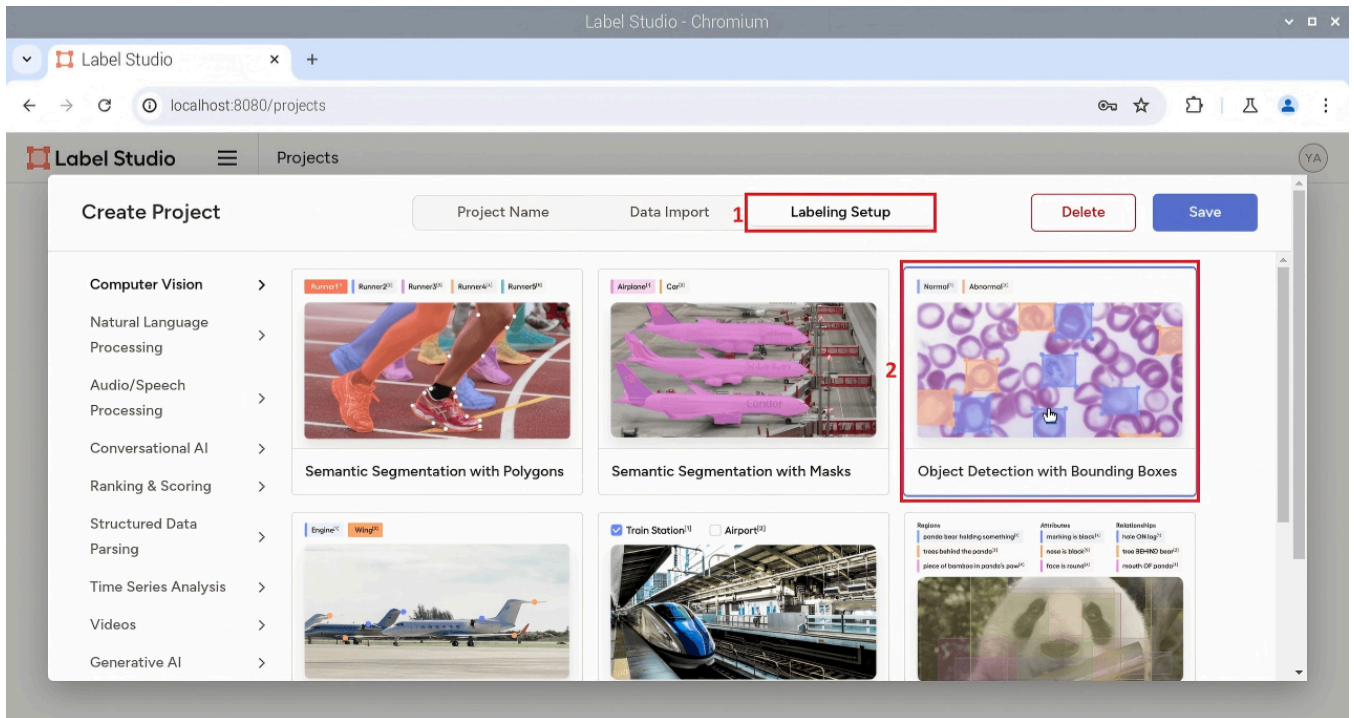
3.3 Import Training Set

Manually import the prepared dataset: Do not import too many images at once. It is recommended to select 50-100 images each time, then import multiple times.

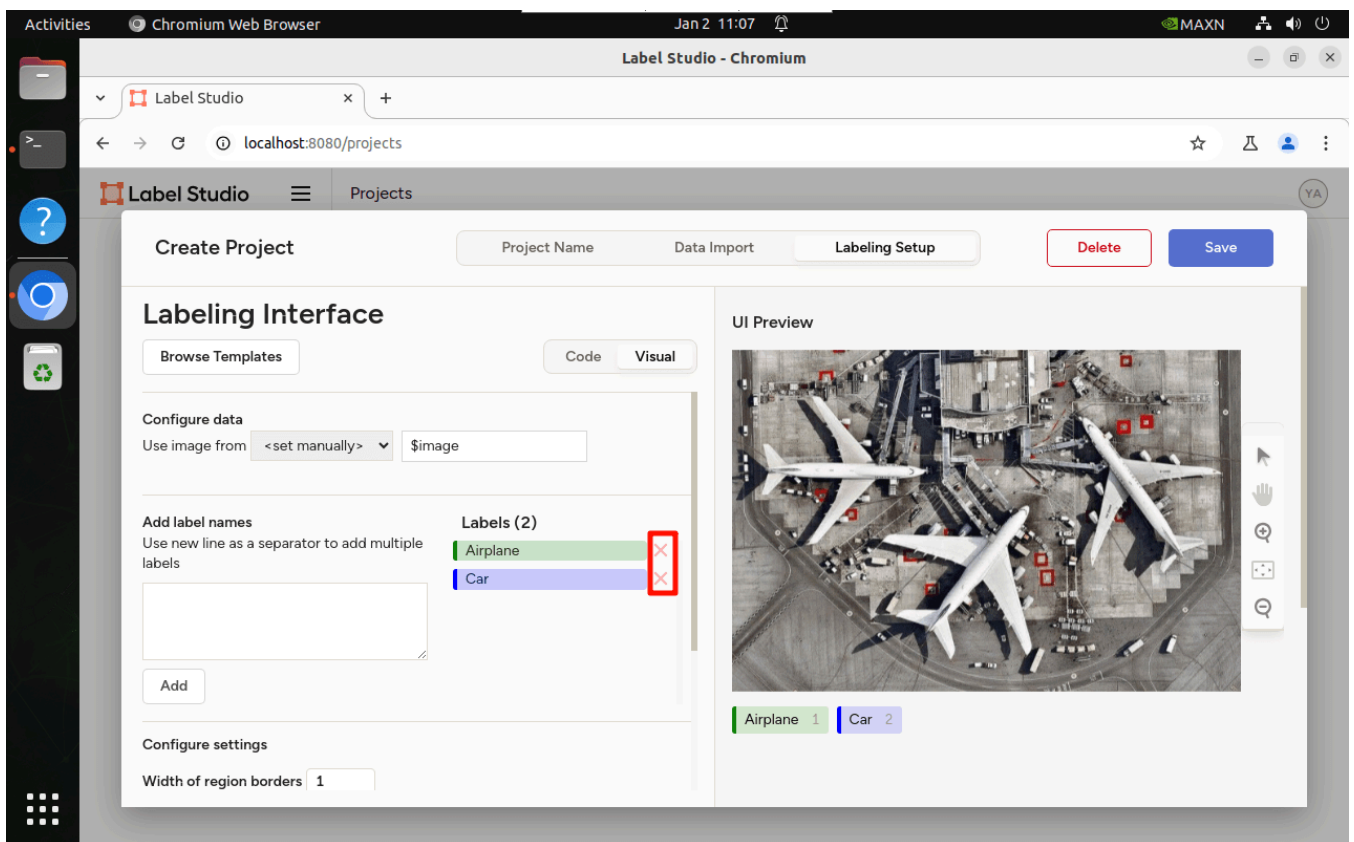


3.4 Label Settings

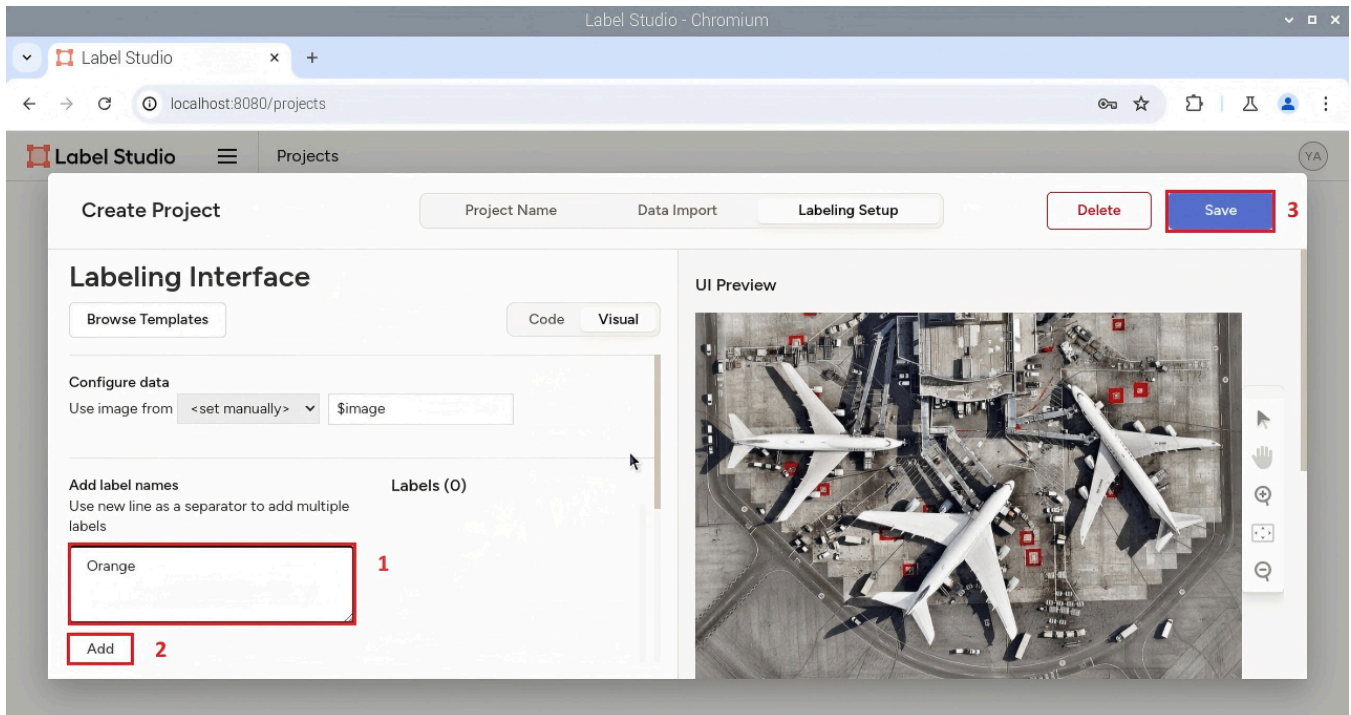
Since we are demonstrating orange recognition, select "Object Detection".



Delete built-in labels:

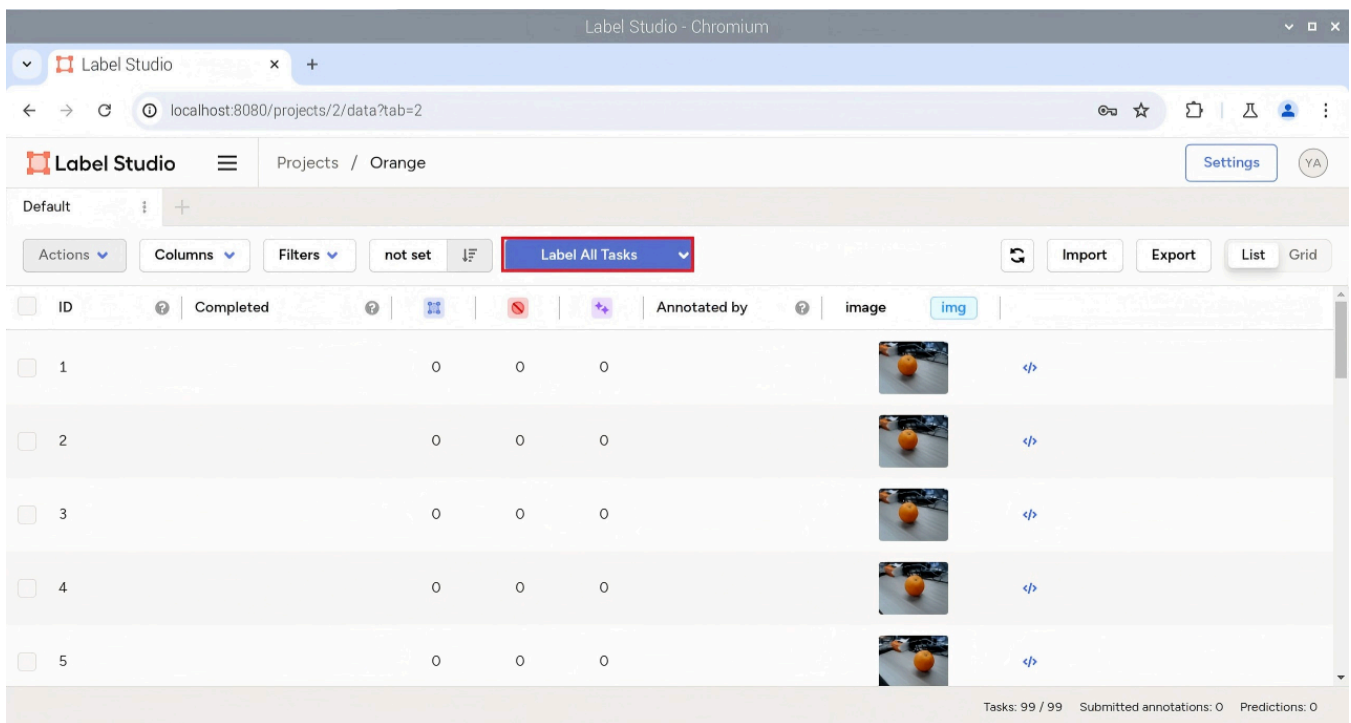


Add new labels:

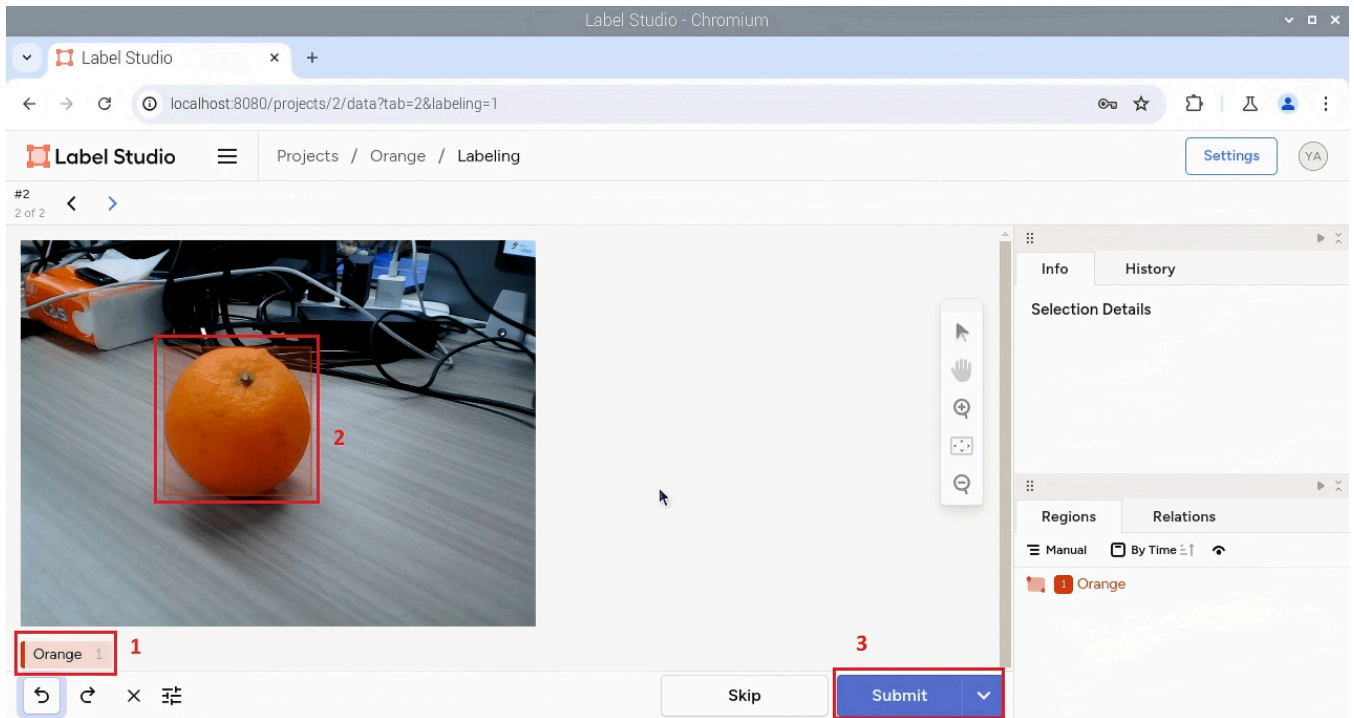


3.5 Dataset Annotation

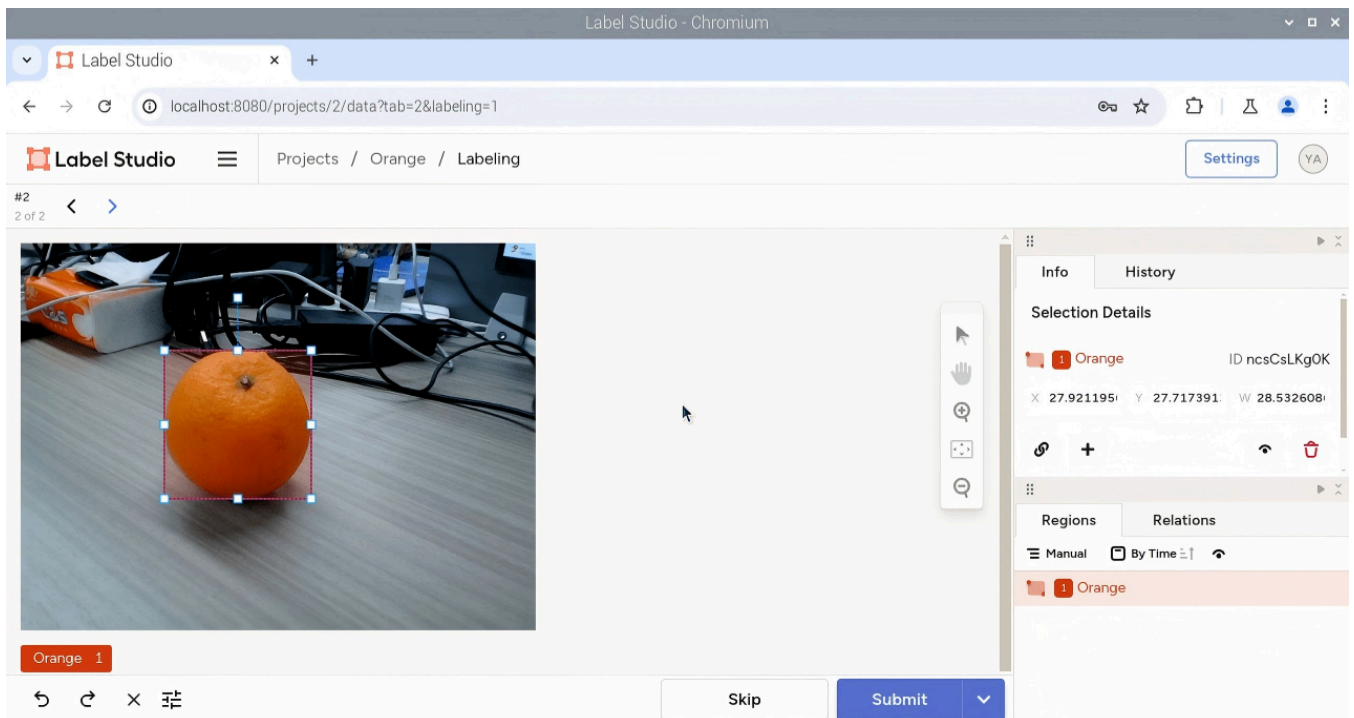
Click "Annotate all tasks":



Annotation steps: First click the "Orange" label, then use the mouse to draw a bounding box around the corresponding orange in the image. After completion, click Submit to proceed to the next annotation.

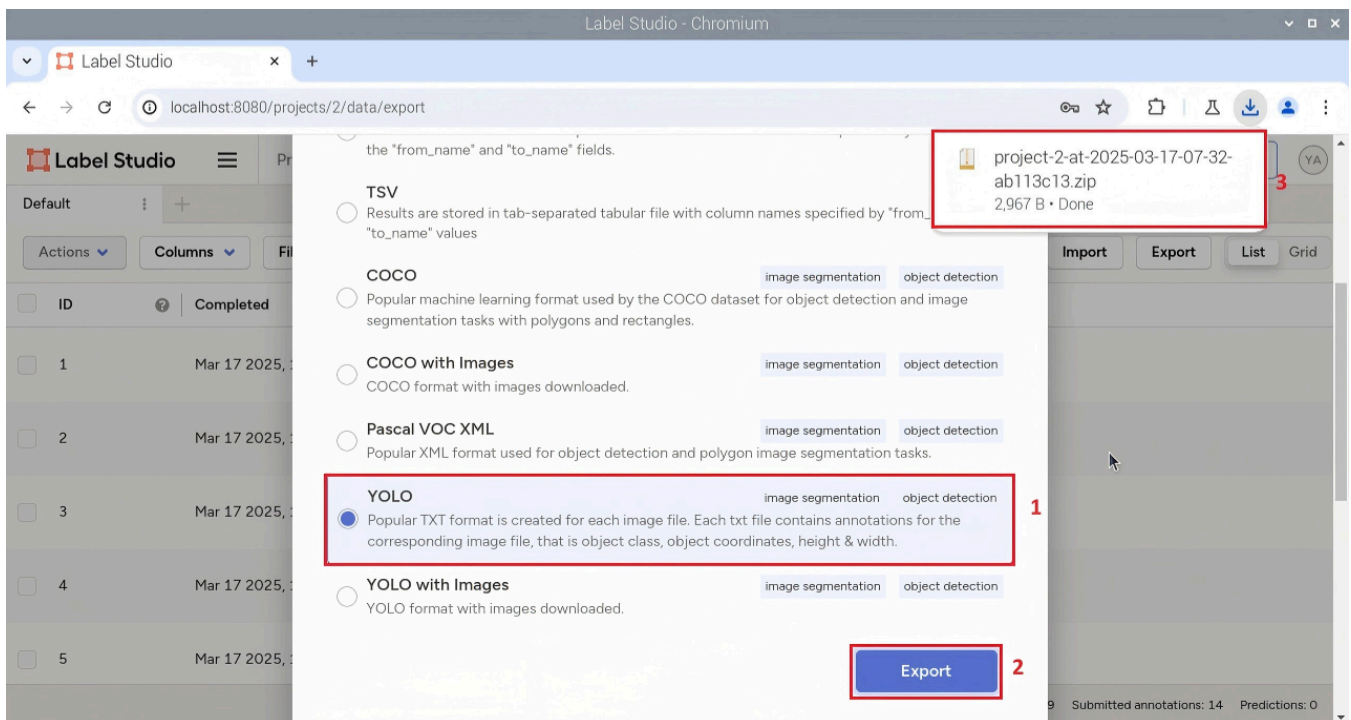
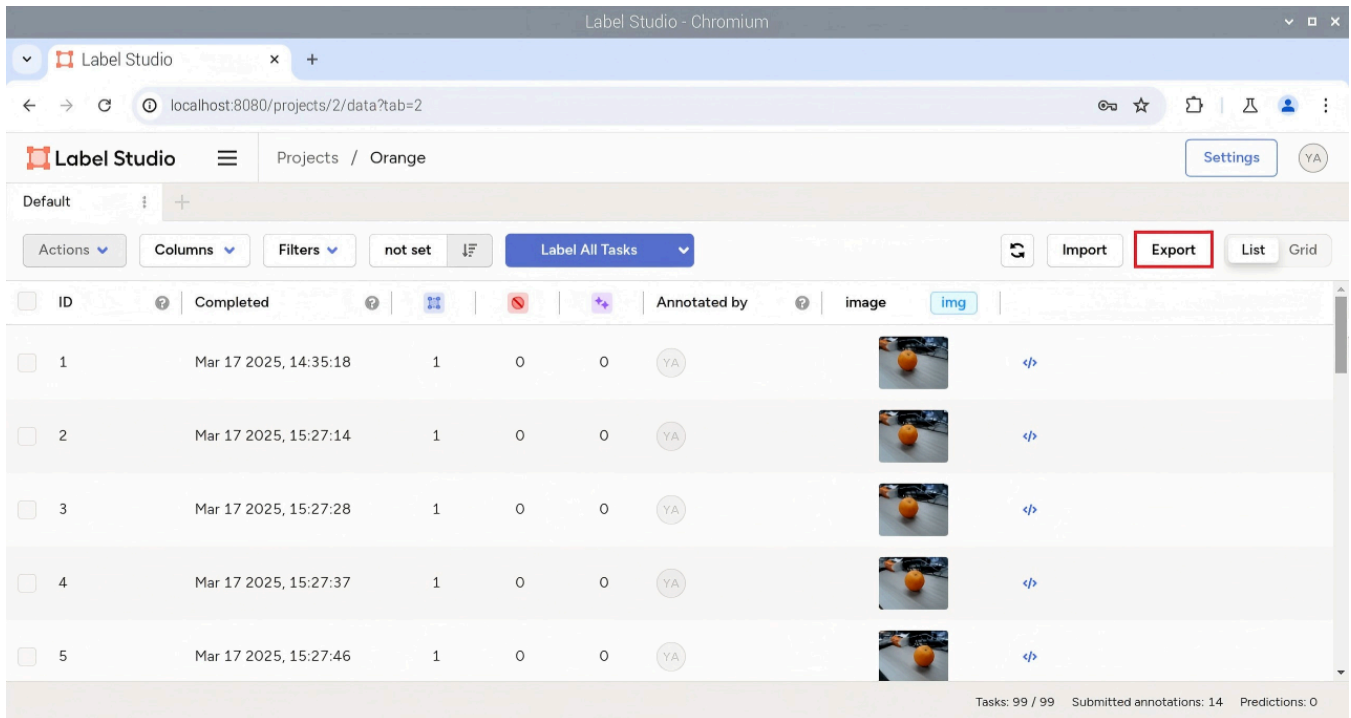


If the bounding box does not completely enclose the orange, you can click on the bounding box to adjust the position of each edge:



3.6 Export Dataset

Select YOLO format to export. The browser will automatically download a compressed package.



3.7 Dataset Directory Structure

The directory structure corresponding to the compressed package exported from Label Studio:

```
.
├── images
├── labels
├── classes.txt
└── notes.json
```

You need to split the images and labels exported from Label Studio into train (training set) and val (validation set), with an approximate ratio of 8:2. The images and labels in both train and val need to have matching names.

```
.
├── classes.txt
├── notes.json
├── train
│   ├── images
│   └── labels
└── val
    ├── images
    └── labels
```

3.8 Dataset Configuration File

Contains paths to the training set and validation set, as well as the number and names of classes.

```
path: /home/sunrise/ultraanalytics26/data/yahboom_data/orange_data
train: train/images
val: val/images

nc: 1

# Classes
names:
  0: Orange
```

References

<https://github.com/HumanSignal/label-studio>